

# EX2: Alpha Particle Range in Air

## Alpha Prelab

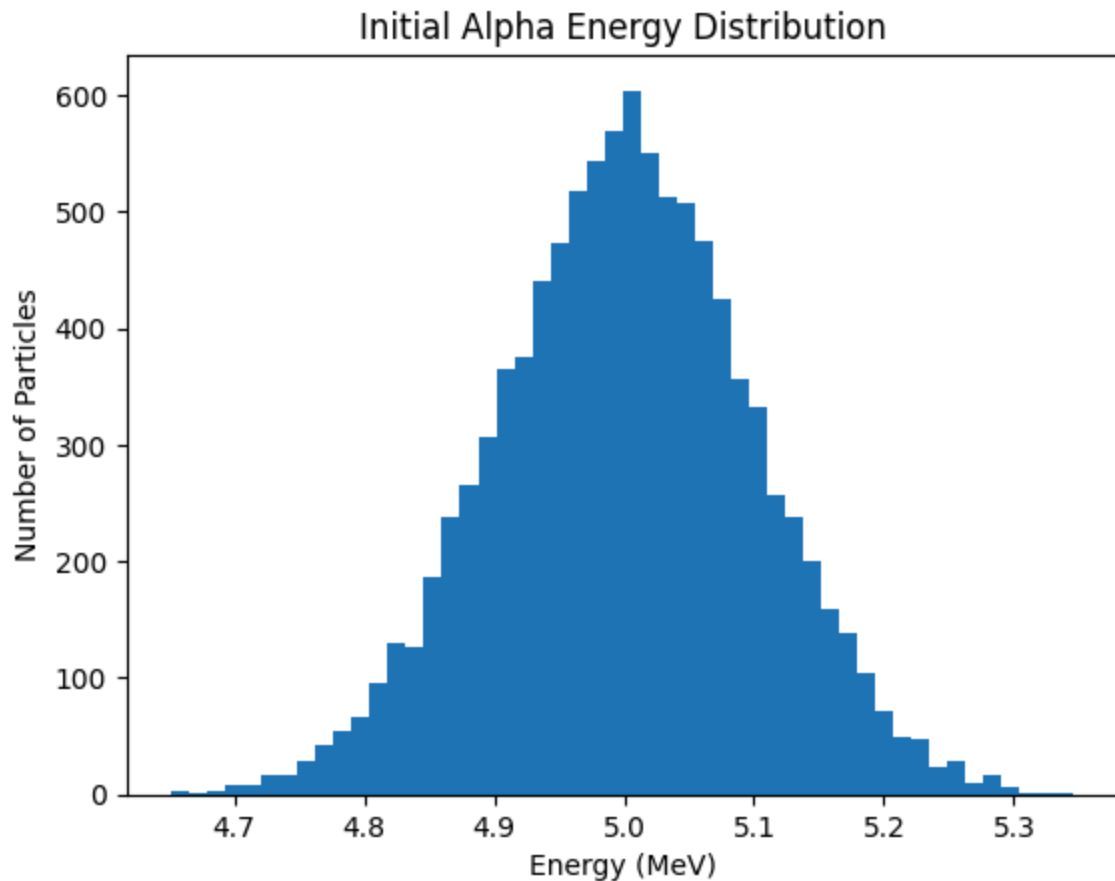
Simulating passage of alpha particles through air. We do so by defining an interaction policy for a single thin layer of air and repeating for  $N$  layers. The policy is as follows: particles can either interact once (which causes it to lose some amount of energy  $dE$ ) OR pass right through. The input of this simulated column of air is a population of particles with a Gaussian distribution of energies centered at  $E_0$  and some spread  $\sigma$ .

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt

# Parameters
E0 = 5.0      # Initial midpoint of Gaussian distribution
sigma = 0.1   # Initial spread/S.D.
minE = 0.3    # Lower bound of E, considered stopped below this point
F = 0.05      # 10% chance of interaction per layer
dE = 0.1      # Energy lost per interaction
```

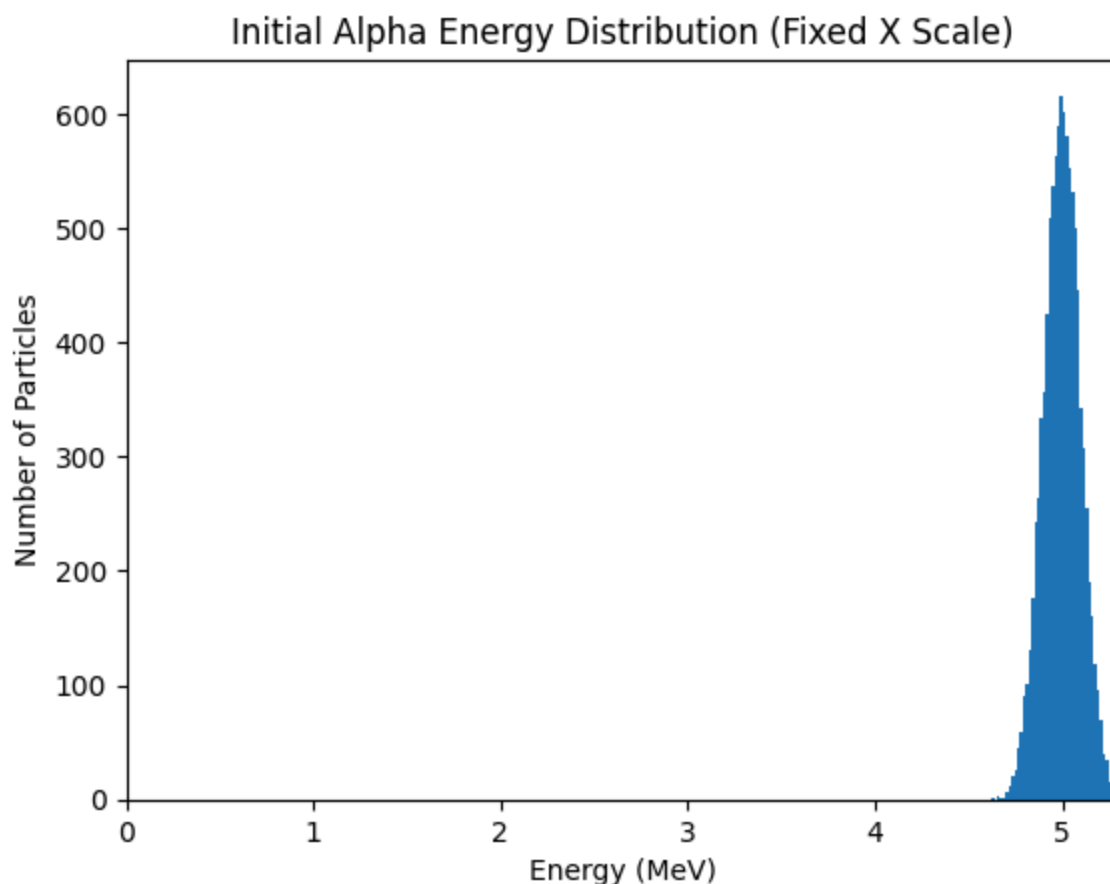
```
In [46]: # Plot initial distribution of energies
particles = np.random.normal(E0, sigma, 10000)

plt.hist(particles, bins=50)
plt.title("Initial Alpha Energy Distribution")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.show()
```



```
In [53]: # Plot initial distribution of energies (with fixed x-axis range)
particles = np.random.normal(E0, sigma, 10000)

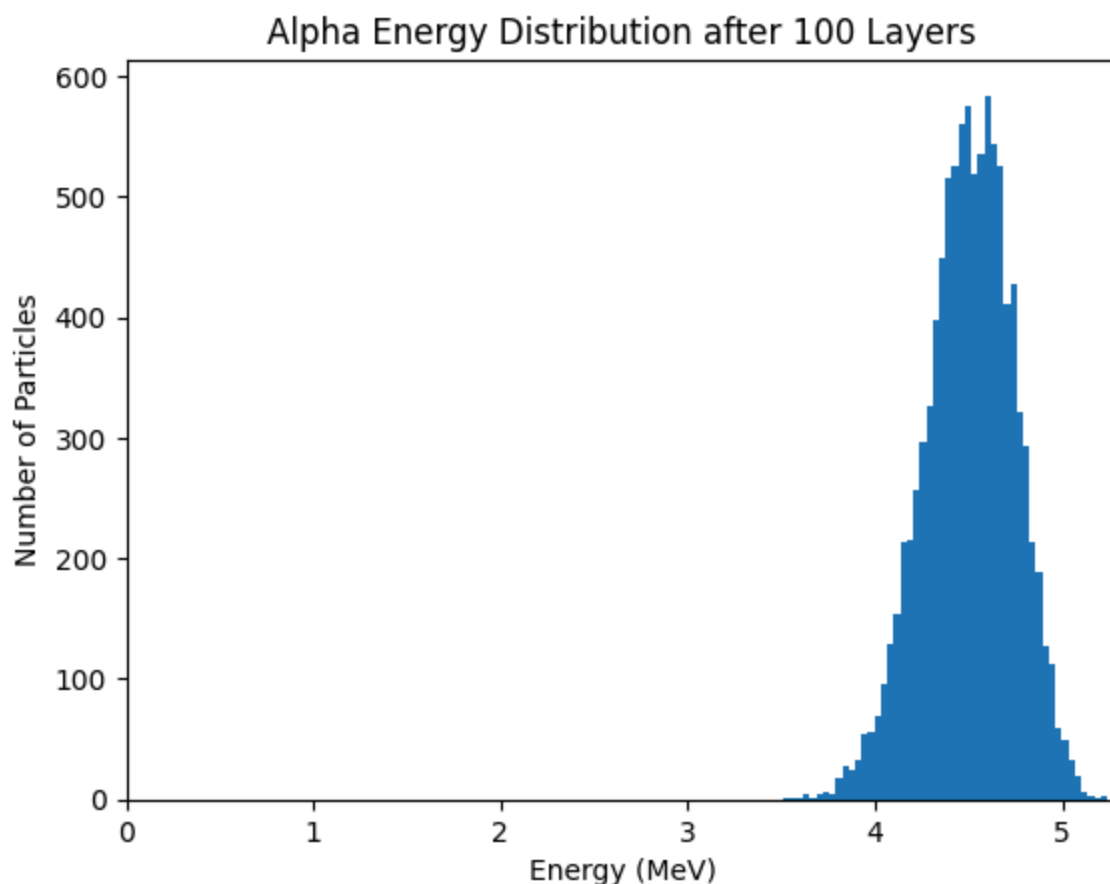
plt.hist(particles, bins=50)
plt.title("Initial Alpha Energy Distribution (Fixed X Scale)")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



```
In [50]: # Reset particle energies
particles = np.random.normal(E0, sigma, 10000)
# Plot energies after 100 layers
layers = 100

for layer in range(layers):
    mask = particles > minE # Only process active particles
    interactions = np.random.random(len(particles[mask])) < F
    particles[mask] -= interactions * dE

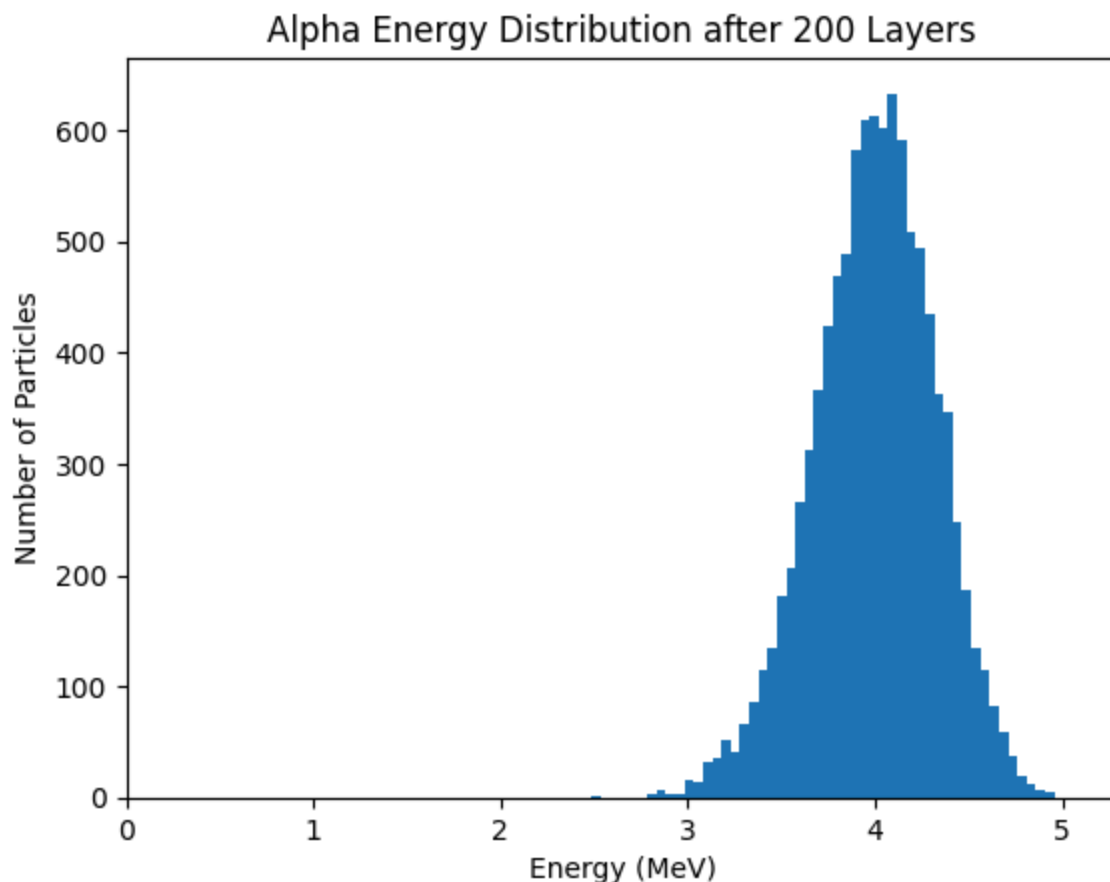
# Plotting
plt.hist(particles[particles > minE], bins=50)
plt.title(f"Alpha Energy Distribution after {layers} Layers")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



```
In [54]: # Reset particle energies
particles = np.random.normal(E0, sigma, 10000)
# Plot energies after 200 layers
layers = 200

for layer in range(layers):
    mask = particles > minE # Only process active particles
    interactions = np.random.random(len(particles[mask])) < F
    particles[mask] -= interactions * dE

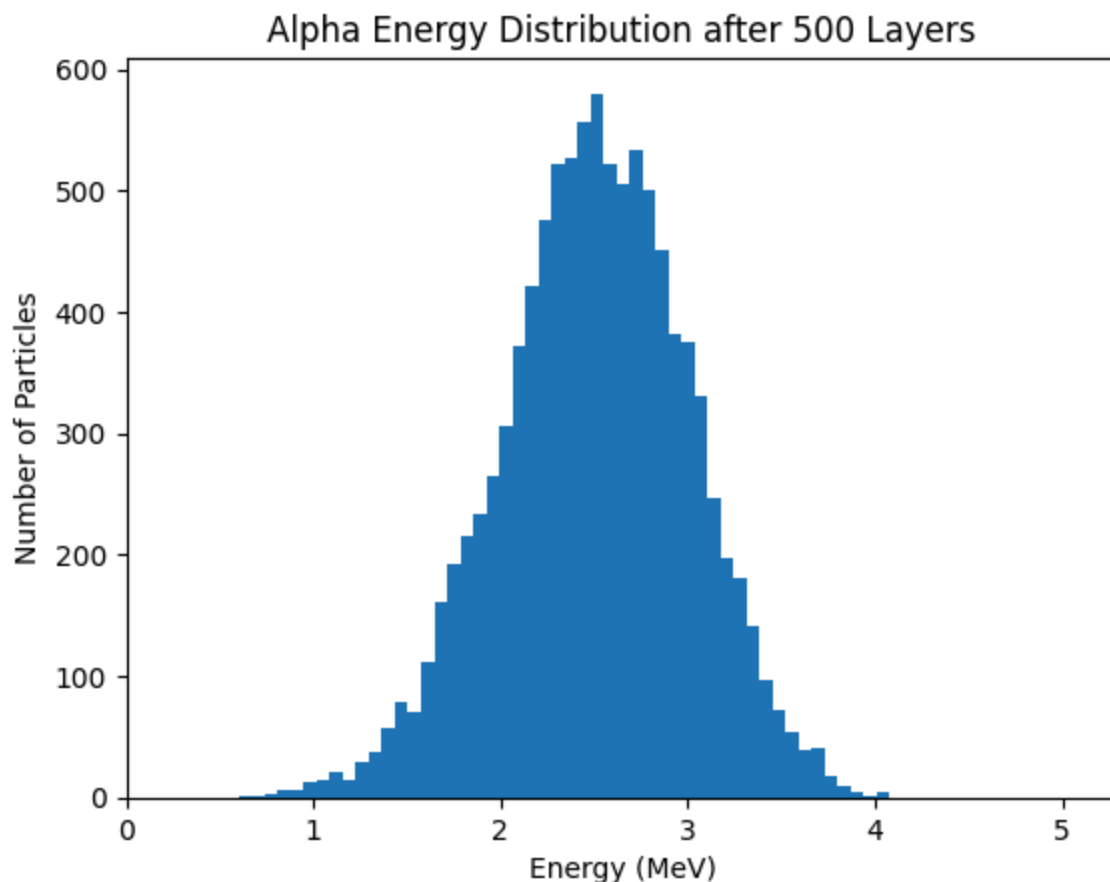
# Plotting
plt.hist(particles[particles > minE], bins=50)
plt.title(f"Alpha Energy Distribution after {layers} Layers")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



```
In [57]: # Reset particle energies
particles = np.random.normal(E0, sigma, 10000)
# Plot energies after 500 layers
layers = 500

for layer in range(layers):
    mask = particles > minE # Only process active particles
    interactions = np.random.random(len(particles[mask])) < F
    particles[mask] -= interactions * dE

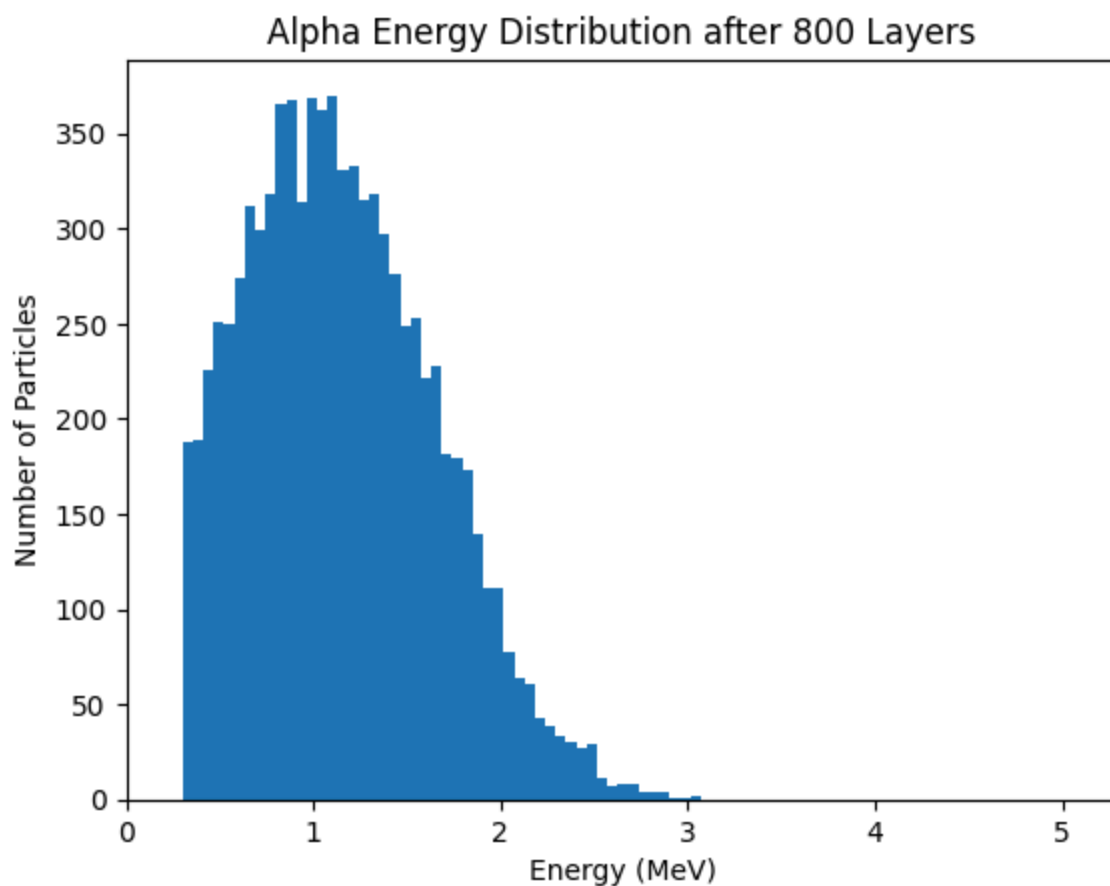
# Plotting
plt.hist(particles[particles > minE], bins=50)
plt.title(f"Alpha Energy Distribution after {layers} Layers")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



```
In [58]: # Reset particle energies
particles = np.random.normal(E0, sigma, 10000)
# Plot energies after 800 layers
layers = 800

for layer in range(layers):
    mask = particles > minE # Only process active particles
    interactions = np.random.random(len(particles[mask])) < F
    particles[mask] -= interactions * dE

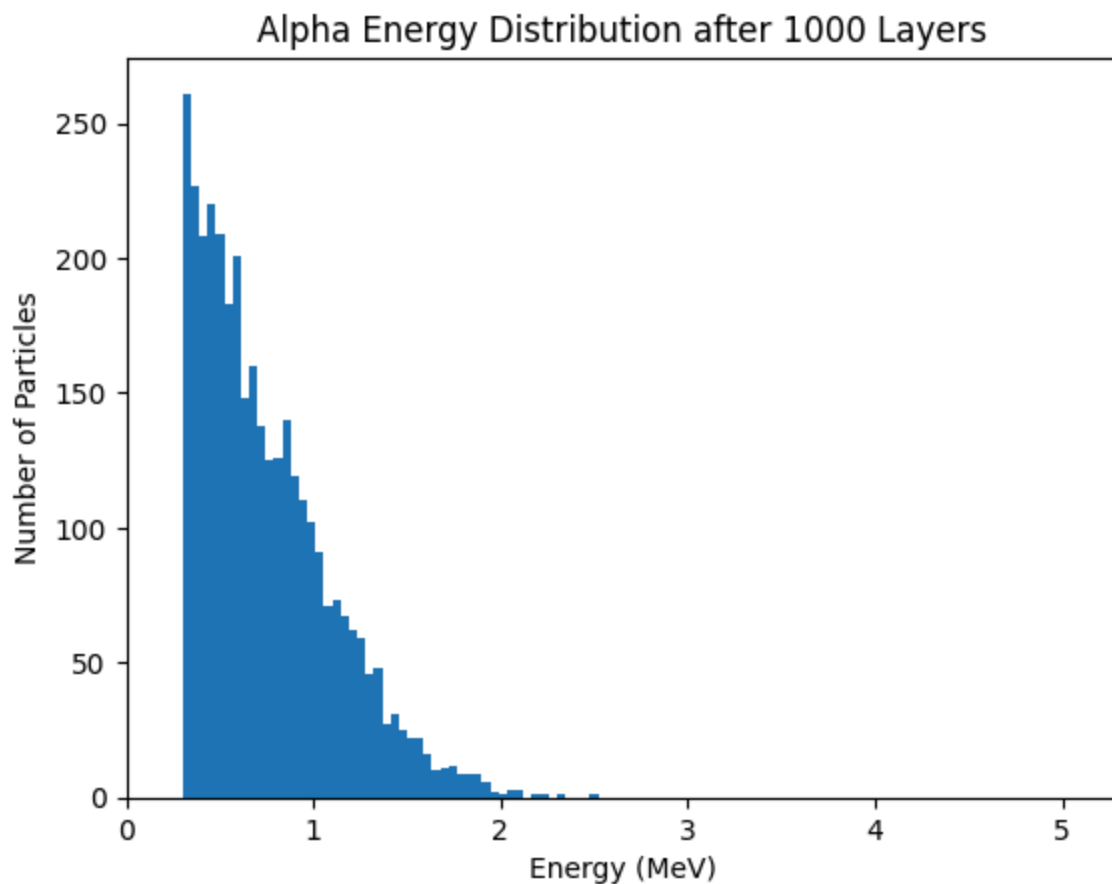
# Plotting
plt.hist(particles[particles > minE], bins=50)
plt.title(f"Alpha Energy Distribution after {layers} Layers")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



```
In [60]: # Reset particle energies
particles = np.random.normal(E0, sigma, 10000)
# Plot energies after 1000 layers
layers = 1000

for layer in range(layers):
    mask = particles > minE # Only process active particles
    interactions = np.random.random(len(particles[mask])) < F
    particles[mask] -= interactions * dE

# Plotting
plt.hist(particles[particles > minE], bins=50)
plt.title(f"Alpha Energy Distribution after {layers} Layers")
plt.ylabel("Number of Particles")
plt.xlabel("Energy (MeV)")
plt.xlim(0, 5 + 3 * sigma) # 3 SD = 99.7% of data
plt.show()
```



The overall trend I'm noticing: as the layers of air  $N$  increases, the midpoint shifts left and the distribution also spreads out further. This makes sense since more layers of air means higher probability of interaction which means particles will keep losing energy until majority of them just stop.